MEDSYS Healthcare Platform
Client Specification and Implementation Review

Standalone handover review document prepared for technical management review.

Project Code	MEDLINK-Version2
Audience	All Medical Field Professionals and Technical Management
Document Version	1.2
Date	April 6, 2026
Status	Tech Manager Review Draft
Branding	Reserved space for logo and branding

This PDF is intentionally separate from the backend API client specification so it can be reviewed as an independent management handover artifact.

1. Executive Summary

MEDSYS is a role-based healthcare operations platform for clinic and medical-center workflows. The current implementation supports patient management, appointment and walk-in consultation flows, diagnosis and medical test search, prescription lifecycle handling, assistant-driven dispensing, inventory search and stock deduction, user management, multi-role identity, analytics, and audit logging.

The strongest implementation area today is the doctor and assistant operational flow. Doctors can save a complete consultation through a single workflow endpoint, appointment and walk-in modes are both supported, prescription items are operationally split into `clinical` and `outside`, only clinical items enter the dispense queue, and inventory now supports alerts, packaging-conversion metadata, movement conversion responses, stock-adjust endpoints, batch tracking, and reporting summaries. Patient history is also recorded structurally through encounters, timeline events, narrative history notes, and long-term conditions.

2. Document Control

Field	Value
Document title	MEDSYS Client Specification and Implementation Review
Project code	MEDLINK-Version2
Version	1.2
Date	April 6, 2026
Status	Tech Manager Review Draft
Basis	Current repository implementation and verified API behavior

3. System Overview

1. Frontend application for doctor, assistant, owner, analytics, and support-facing workflows.
2. BFF/API access layer for authenticated browser integration.
3. Core backend API with implemented `/v1/*` business routes.
4. Persistence and platform services including PostgreSQL, audit logging, search integration, typed validation, and optional queue support.

Main Implemented Backend Domains

- Authentication and session management
- Users and permissions
- Patients, families, guardian handling
- Appointments and visit start
- Consultations and encounters
- Prescriptions and dispense workflow
- Inventory and stock movements
- Clinical terminology lookup
- Analytics and observability
- Audit logs

4. Core Functional Scope

Authentication and Access Control

- Login, refresh, logout, and current-user session are implemented.
- Effective permission model uses assigned roles plus extra permissions.
- Multi-role identity is implemented with `roles`, `active_role`, and `workflow_profiles`.
- Authenticated active-role switching is implemented through `POST /v1/auth/active-role`.
- Sensitive action throttling is in place.
- Frontend should use returned permissions for actions, `active_role` for workspace selection, and `workflow_profiles` for role-specific behavior.

Patient Management

- Patient search and quick selection are implemented.
- Patient summary list and full profile are implemented.
- Patient creation, update, guardian-aware flows, family auto-creation, allergies, conditions, history, timeline, vitals, bundled family detail in profile, and consultation-centric history are implemented.
- `dateOfBirth` is canonical; `age` is only a helper.

Consultation Workflows

- One workflow endpoint: `POST /v1/consultations/save`.
- Supports `workflowType = appointment` and `workflowType = walk_in`.
- Supports `patientId`, `patientDraft`, and `guardianDraft`.
- Supports diagnoses, tests, vitals, allergies, notes, clinical summary, and prescription in one save.

Prescription and Dispense

- Prescription creation within consultation save is implemented.
- `clinical` vs `outside` source behavior is implemented.
- Pending dispense queue, doctor direct dispense, assistant queue-based dispense, stock deduction, and inventory movement creation are implemented.
- Inventory search exists for unresolved clinical dispense items.

Inventory and Stock Control

- Inventory item creation and update are implemented.
- Inventory alerts and reorder suggestions are available through `GET /v1/inventory/alerts`, and operational summaries are available through `GET /v1/inventory/reports`.
- Inventory quantity changes should use movement routes, stock corrections should use `POST /v1/inventory/:id/adjust-stock`, and `PATCH /v1/inventory/:id` remains for item details, thresholds, and packaging rules.
- `unit` is the base stock unit, while `dispenseUnit` / `dispenseUnitSize` and `purchaseUnit` / `purchaseUnitSize` describe packaging conversions. Batch-level tracking is available through `GET /v1/inventory/:id/batches` and `POST /v1/inventory/:id/batches`.

5. Workflow Model

Appointment Mode

- Patient already exists and appointment already exists.
- Backend validates appointment and patient match.
- Backend stores encounter, vitals, diagnoses, tests, allergies, history, and prescription.
- If no clinical prescription items exist, workflow can complete immediately.
- If clinical items exist and are not directly dispensed, workflow stays doctor-finished until dispense is completed.

Walk-In Mode

- No appointment is required.
- Doctor searches patient and can quick-create inline using `patientDraft`.
- Backend resolves existing patient or creates a new one, reuses or creates visit context, then creates encounter and related clinical data.
- Backend creates timeline event and optional history entry, and can promote flagged diagnoses into patient conditions.

> **Implemented conflict protection:** if an active walk-in consultation already exists for the patient, backend now returns `409 Conflict` with a clear frontend-safe message instead of a generic server failure.

Prescription Source Logic

- `clinical` items affect workflow completion, create pending dispense queue if not directly dispensed, and require inventory resolution.
- `outside` items remain in prescription print/history, do not create dispense queue, and do not block completion.

6. API Integration Model

Area	Implemented Endpoint
Auth	`/v1/auth/login`, `/v1/auth/refresh`, `/v1/auth/me`, `/v1/auth/logout`, `/v1/auth/active-role`
Patient search	`/v1/search/patients`
Patients	`/v1/patients`, `/v1/patients/:id/profile`, `/v1/patients/:id/consultations`
Appointments	`/v1/appointments`
Visit start	`/v1/visits/start`
Consultation workflow	`/v1/consultations/save`
Prescriptions	`/v1/prescriptions`, `/v1/prescriptions/:id/dispense`
Dispense queue	`/v1/prescriptions/queue/pending-dispense`
Inventory	`/v1/inventory`, `/v1/inventory/:id`, `/v1/inventory/search`, `/v1/inventory/alerts`, `/v1/inventory/reports`, `/v1/inventory/:id/movements`, `/v1/inventory/:id/adjust-stock`, `/v1/inventory/:id/batches`
Clinical lookup	`/v1/clinical/diagnoses`, `/v1/clinical/tests`, `/v1/clinical/diagnoses/:code/recommended-tests`

Frontend Contract Direction

- Diagnosis field = diagnoses only.
- Medical Tests field = test catalog only.
- Selected diagnosis can request suggested tests below.
- Consultation save should be one workflow payload, not many independent save steps.
- Dispense completion requires real `inventoryItemId` selection for clinical items.
- Inventory edit should be used for descriptive fields, thresholds, packaging rules, and supplier metadata.
- Inventory quantity changes should be sent as movements (`in`, `out`, `adjustment`) with optional `movementUnit`, not as direct stock rewrites.
- Use `POST /v1/inventory/:id/adjust-stock` for “actual counted stock is X” corrections and `POST /v1/inventory/:id/batches` when receiving a new batch or lot.

Inventory Actions In Simple Terms

- **Edit Item Details**: change the item name, supplier, packaging setup, minimum stock level, and notes.
- **Add Stock**: record newly received stock.
- **Remove Stock**: record stock that was used, dispensed, damaged, or expired.
- **Correct Stock Count**: set the system to the real counted quantity when physical stock and system stock do not match.
- **Add Batch**: record a new lot or batch with its own expiry date and storage location.

Operational Language Guide

- `movementUnit` should be shown to staff as **Stock Unit Used**.
- `adjust-stock` should be shown to staff as **Correct Stock Count**.
- `stockSummary.currentStock` should be shown as **Current Stock**.
- `stockSummary.minimumStock` should be shown as **Minimum Stock Level**.

- `stockSummary.shortageToMinimum` should be shown as **Short by**.
- `stockSummary.purchasePackEquivalent` should be shown as **Equivalent Purchase Packs**.
- `stockSummary.dispensePackEquivalent` should be shown as **Equivalent Dispense Packs**.
- `inventory reports` should be described as **Inventory Summary and Warnings**.

Simple Staff Examples

- **Current Stock:** 200 tablets
- **Minimum Stock Level:** 50 tablets
- **Short by:** 10 tablets
- If 2 boxes are received and 1 box = 100 tablets, the system adds 200 tablets automatically.
- If physical counted stock is 180, staff should use **Correct Stock Count** instead of manually calculating the difference.

7. Error Handling Contract

Endpoint	Status	Meaning
`POST /v1/consultations/save`	400	Validation or input issue
`POST /v1/consultations/save`	404	Patient, appointment, or guardian not found
`POST /v1/consultations/save`	409	Active walk-in conflict or appointment/patient mismatch
`POST /v1/prescriptions/:id/dispense`	400	Missing or invalid dispense item payload
`POST /v1/prescriptions/:id/dispense`	404	Prescription or inventory item not found
`POST /v1/prescriptions/:id/dispense`	409	Already dispensed or insufficient stock
`POST /v1/prescriptions/:id/dispense`	429	Sensitive action rate limit exceeded
`GET /v1/clinical/tests`	503	Terminology provider unavailable

```
{
  "message": "Active walk-in consultation already exists for this patient. Complete or dispense the current
consultation before starting a new one."
}
```

8. Current Implementation Status

- Consultation workflow endpoint implemented and verified.
- Appointment and walk-in branching implemented and verified.
- Guardian linking and guardian draft creation implemented and verified.
- Patient timeline and history integration implemented and verified.
- Diagnosis persistence into patient conditions implemented and verified.
- Patient profile now bundles family detail, and consultation history can be loaded from one encounter-grouped patient endpoint.
- Terminology lookup endpoints implemented and verified.
- Pending dispense queue enrichment implemented and verified.
- Inventory search for assistant stock matching implemented and verified.
- Inventory alerts, packaging conversion fields, stock-movement-only quantity updates, stock-adjust workflow, batch tracking, and supplier/expiry reporting are implemented and verified.
- Source-aware prescription workflow implemented and verified.
- Walk-in conflict handling with clean `409` implemented and verified.
- Swagger request and error response documentation updated.

Verification Status

- `npm run typecheck` passing
- `npm run test --workspace @medsys/api` passing
- 92 API tests passing at the time of this document

9. Known Limitations and Future Scope

- Imaging and procedures are not yet part of the main clinical test catalog.
- Prescription items do not yet persist inventory linkage at item-create time.
- Advanced clinical decision support remains limited.
- Compliance maturity is operationally helpful but not certification-ready.
- External lab and imaging integration remain future scope.

10. Handover Recommendation

The most important achievements so far are backend workflow consolidation around consultation save, reduction of doctor UX from many steps to one final save, proper appointment versus walk-in lifecycle separation, source-based prescription workflow, assistant queue and inventory resolution alignment, and clearer frontend error contracts with Swagger accuracy.

1. Validate the workflow model against real clinic operations.
2. Confirm the current status names and handoff stages fit business language.
3. Review remaining terminology and integration scope for imaging and external systems.
4. Approve the current API contract as the baseline for frontend completion.